

Tiling Game

Barchin y Charos están jugando a un juego en una cuadrícula de $2N \times 2M$ celdas cuadradas de una unidad. Las filas están numeradas del 0 al $2N - 1$ de arriba abajo, y las columnas están numeradas del 0 al $2M - 1$ de izquierda a derecha. Para $0 \leq i < 2N$ y $0 \leq j < 2M$, denotamos la celda de la fila i y la columna j como (i, j) .

Barchin le da a Charos **bloques** de $N \cdot M$, uno por uno. Cada bloque es un cuadrado de 2×2 formado por cuatro **fichas** de 1×1 . Barchin ha pintado cada ficha del bloque de negro o de blanco, garantizando que **al menos una ficha sea blanca**.

Charos debe colocar cada bloque en la cuadrícula **inmediatamente después de recibirlo**, sin saber qué bloques recibirá más adelante. Los bloques no se pueden girar. Cada bloque debe colocarse enteramente dentro de la cuadrícula, ocupando exactamente cuatro casillas. Además, la ficha superior izquierda de cada bloque debe ocupar una casilla cuyas coordenadas de fila y columna sean ambas pares. Cada casilla de la cuadrícula puede estar ocupada por un máximo de un bloque.

Barchin gana la partida si, tras colocar cualquier bloque, existe un cuadrado de casillas 2×2 cubierto por cuatro fichas negras. Formalmente, si las casillas (a, b) , $(a + 1, b)$, $(a, b + 1)$ y $(a + 1, b + 1)$ están todas cubiertas por fichas negras para algún $0 \leq a < 2N - 1$ y $0 \leq b < 2M - 1$, entonces Barchin gana. No es necesario que los índices a y b sean pares.

Charos gana si coloca todos los bloques $N \cdot M$ sin que Barchin gane en ningún momento. Ten en cuenta que colocar los bloques $N \cdot M$ cubrirá completamente la cuadrícula.

Tu tarea consiste en implementar una estrategia para que Charos gane la partida. Se puede demostrar que, bajo las restricciones dadas, Charos siempre puede colocar los bloques de manera que se garantice la victoria, independientemente de los colores de los bloques que reciba posteriormente.

Implementation Details

Deberías implementar dos procedimientos:

```
void init(int N, int M)
```

- N : la mitad del número de filas de la cuadrícula.

- M : la mitad del número de columnas de la cuadrícula.
- El procedimiento se ejecuta exactamente una vez por cada caso de prueba, al inicio de la ejecución de tu programa.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR)
```

- TL, TR, BL, BR : los colores de las casillas superior izquierda (top-left), superior derecha (top-right), inferior izquierda (bottom-left) e inferior derecha (bottom-right) del bloque actual, respectivamente, tal y como se muestra en la figura siguiente. Cada valor es 0 (blanco) o 1 (negro).
- Este procedimiento se ejecuta exactamente $N \times M$ veces por caso de prueba, tras la llamada inicial a `init`.



Este procedimiento debe devolver un par de números enteros (i, j) , donde i es la coordenada de la fila y j es la coordenada de la columna de la celda en la que debe colocarse la ficha superior izquierdo de este bloque. Tanto i como j deben ser pares, y la región 2×2 cubierta por el bloque no debe superponerse con ningún bloque colocado anteriormente.

Si `receive_block` devuelve en algún momento un par que no cumpla estos requisitos, o si tras colocar el bloque un cuadrado de celdas de 2×2 queda completamente cubierto por fichas negras, el evaluador terminará inmediatamente tu programa y el veredicto para el caso de prueba será `Output isn't correct`.

El comportamiento del evaluador **no es adaptativo**. Esto significa que la secuencia de bloques que Barchin le da a Charos está fijada antes de que se llame a `init`.

Restricciones

Para cada bloque, sea S el número de fichas negras entre sus cuatro fichas. Es decir, $S = TL + TR + BL + BR$.

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$ para cada bloque.

Subtasks

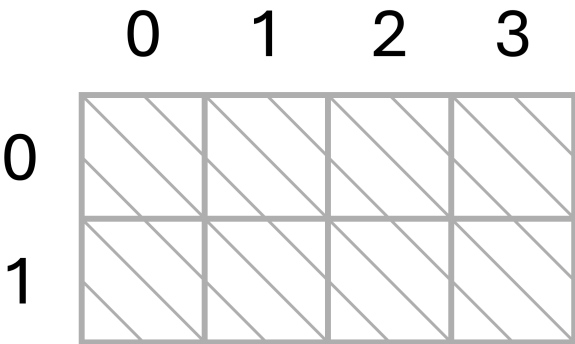
Subtask	Score	Restricciones Adicionales
1	6	$S = 1$ para cada bloque, and $N = 2$.
2	16	$S = 3$ para cada bloque,. $N = M$, N es par, y cada una de las cuatro posibles coloraciones de los bloques aparece exactamente $\frac{N^2}{4}$ veces.
3	10	$S = 1$ para cada bloque.
4	29	$S \leq 2$ para cada bloque.
5	39	No hay restricciones adicionales.

Example

Consideremos un juego con $N = 1$ y $M = 2$, de modo que la cuadrícula tiene 2 filas y 4 columnas. El grader primero llama a:

```
init(1, 2)
```

Inicialmente, todas las casillas están vacías. Así que la cuadrícula tiene este aspecto:



Hay $N \cdot M = 2$ bloques a colocar. Supongamos que Barchin entrega un bloque con tres fichas negras y una ficha blanca en la esquina superior derecha. El grader llama:

```
receive_block(1, 0, 1, 1)
```

Charos decide colocar este bloque en el lado izquierdo de la cuadrícula devolviendo «(0,0)».

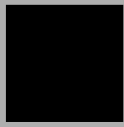
La cuadrícula ahora luce así:

	0	1	2	3
0				
1				

Después, Barchin presenta otro bloque con tres fichas negras y una ficha blanca en la esquina superior izquierda:

```
receive_block(0, 1, 1, 1)
```

La única celda restante con fila par y columna par que puede servir como esquina superior izquierda de un bloque « 2×2 » es $(0, 2)$, por lo que Charos devuelve $(0, 2)$. La cuadrícula queda así:

	0	1	2	3
0				
1				

Ningún cuadrado de « 2×2 » está completamente cubierto por fichas negras, por lo que Charos ha colocado con éxito todas las fichas sin que Barchin haya ganado en ningún momento. Así Charos gana la partida.

Sample Grader

Input format:

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

Output format:

```
R[0] C[0]  
R[1] C[1]  
...  
R[NM-1] C[NM-1]
```

En este caso, $R[k]$ y $C[k]$ son el par de números enteros devueltos por la k -th llamada a `receive_block`.